IPB University
Bogor Indonesia
Program Studi
Sarjana, Magister & Doktor
Statistika & Sains Data

DIKTISAINTEK BERDAMPAK

STA1381 – Pengantar Sains Data

Pertemuan-6

Seleksi Peubah: Metode Filter

Anang Kurnia
Cici Suhaeni
Septian Rahardiantoro

2026

Outline

- Konsep Seleksi Peubah
- Metode Filter
- Filtering selection variables in text analytics

Konsep Seleksi Peubah

- Seleksi peubah (variable selection) adalah proses memilih sebagian peubah (fitur) yang paling relevan dari sekumpulan peubah dalam dataset untuk digunakan dalam pemodelan, dengan tujuan meningkatkan kinerja model, mengurangi kompleksitas, dan menghindari overfitting.

- Seleksi peubah bukan sekadar "mengurangi jumlah peubah", tetapi:
  - Mencari kombinasi peubah yang paling informatif untuk memprediksi target.

Manfaat Seleksi Peubah

1. Mengurangi Overfitting: Terlalu banyak peubah → model bisa "menghafal" data latih
2. Meningkatkan Akurasi dan Stabilitas Model: Seleksi peubah membantu model fokus pada sinyal yang benar-benar penting
3. Mengurangi Dimensi
4. Mempercepat Komputasi
5. Meningkatkan Interpretabilitas: Model dengan 5–10 peubah penting jauh lebih mudah dijelaskan dibanding model dengan 200 peubah.
6. Mengurangi Biaya Pengumpulan Data

Ilustrasi pada pemodelan

- Misalkan dalam model linear

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon$$

  yang umumnya digunakan untuk menggambarkan hubungan antara peubah respon Y dan sekumpulan peubah prediktor $X_1, X_2 \ldots, X_p$.

- Umumnya, model linier ini dapat diduga dengan metode kuadrat terkecil (OLS).

- Dugaan model dengan OLS berprinsip untuk meminimumkan jumlah kuadrat galat.

- Sifat penduga OLS:
  - Nilai dugaan parameternya bersifat tidak berbias
  - Nilai dugaan parameternya memiliki ragam terkecil jika dibandingkan dengan penduga lainnya

- Oleh karenanya, penduga OLS pada model regresi linier merupakan penduga parameter tak bias terbaik.

- Namun, banyak orang yang beralih tidak menggunakan OLS dalam pendugaan parameter untuk pemodelan regresi linier.

- Beberapa alasan:
  - **Keakuratan Prediksi**: penduga OLS tidak lagi bersifat tak bias terbaik ketika $p \gg n$, bahkan OLS tidak dapat digunakan sama sekali → gagal dalam konteks prediksi.
  - **Kemudahan Interpretasi Model**: penduga OLS akan menghasilkan dugaan setiap parameter di dalam model, meskipun pada peubah prediktor yang tidak berhubungan dengan respon. Adanya dugaan yang tidak relevan tersebut, akan membuat interpretasi model lebih kompleks (rumit).

- Dalam rangka meningkatkan kemudahan dalam interpretasi modelnya, metode yang digunakan adalah: *feature selection* atau *variable selection*
  - *feature selection* atau *variable selection* → menyisihkan peubah prediktor yang tidak relevan untuk digunakan pada model

Metode Seleksi Peubah

1. Filter method

- Memilih fitur berdasarkan statistik murni, tanpa melibatkan algoritma machine learning tertentu.
- Fitur dinilai dulu → baru model dilatih
- Setiap fitur diberi skor berdasarkan hubungannya dengan target, misalnya: Chi-Square, ANOVA F-test, Korelasi (Pearson/Spearman), Information Gain, Mutual Information

2. Wrapper method

- Seleksi fitur dilakukan dengan menguji langsung performa model.
- Pilih subset fitur → latih model → evaluasi → ulangi
- Model dilatih berkali-kali untuk mencari kombinasi fitur terbaik: Forward Selection, Backward Elimination, Recursive Feature Elimination (RFE)

3. Embedded method

- Seleksi fitur dilakukan secara otomatis saat proses training model.
- Seleksi fitur sudah menjadi bagian dari algoritma: LASSO, Elastic Net, Decision Tree, Random Forest (feature importance)

Metode Filtering

- Metode filter bekerja dengan menyaring fitur yang kurang penting dan mempertahankan fitur yang memberikan nilai tambah pada model.
- Penentuan pentingnya fitur dilakukan dengan melihat seberapa besar pengaruh fitur tersebut terhadap output atau fitur target.
- Fitur kemudian diberi peringkat berdasarkan tingkat signifikansinya, lalu fitur dengan peringkat tertinggi dipilih sementara yang tidak relevan dihapus.

- Proses pemeringkatan fitur dilakukan untuk menentukan fitur mana yang paling berpengaruh terhadap fitur target.

- Untuk melakukannya, digunakan berbagai uji statistik yang dapat mengukur hubungan antara fitur dan target.
  - Uji-uji ini memberikan peringkat fitur berdasarkan tingkat "kepentingannya" dalam memengaruhi hasil.
  - Beberapa metode statistik yang umum digunakan dalam proyek data science adalah Chi-squared, ANOVA, dan korelasi.

- Pemilihan metode yang tepat bergantung pada jenis fitur (numerik atau kategorik) serta hubungan yang ada di antara keduanya.

- Pada materi ini akan dibahas metode: Chi-squared, ANOVA, dan korelasi

Beberapa metode filtering

1. Chi-squared test

- Uji Chi-square digunakan untuk menentukan ada atau tidaknya hubungan antara dua fitur kategorik, baik biner maupun multikelas.
- Pengujian ini dimulai dengan menetapkan Hipotesis Nol (H0) bahwa tidak ada hubungan antara kedua variabel, serta Hipotesis Alternatif (H1) bahwa terdapat hubungan yang signifikan.

- Metode ini bekerja dengan membandingkan frekuensi yang diamati dengan frekuensi yang diharapkan jika tidak ada hubungan.
- Selanjutnya dihitung nilai p-value untuk menentukan apakah Hipotesis Nol ditolak atau diterima. Jika p-value kecil dan frekuensi observasi sangat berbeda dari frekuensi harapan, maka H0 ditolak dan nilai p-value tersebut digunakan untuk memberi peringkat serta memilih fitur yang paling penting.

Kriteria seleksi fitur:
1. Memilih fitur dengan p-value terkecil
2. Memilih fitur dengan p-value < 0.05
3. Memilih k fitur sesuai urutan

https://colab.research.google.com/drive/1VDVKup7DPwzpP7IMEXLhoyQq7-W7mFfm?usp=sharing

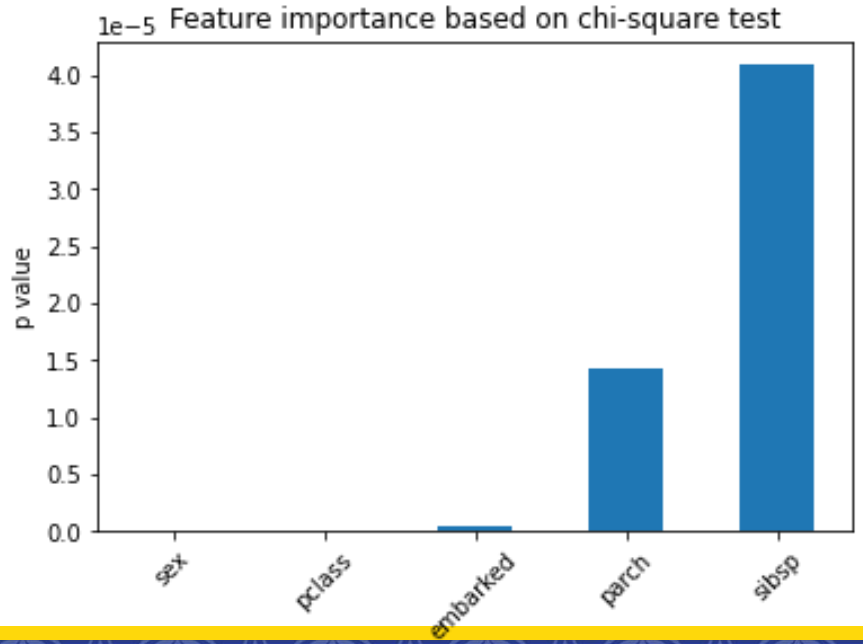
```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
from sklearn.model_selection import train_test_split

variables = ['pclass', 'survived', 'sex', 'sibsp', 'parch', 'embarked']
data = pd.read_csv(
'https://www.openml.org/data/get_csv/16826755/phpMYEkMl',
usecols=variables,
na_values='?',
)
data.dropna(subset=['embarked'], inplace=True)
data.head()

X_train, X_test, y_train, y_test = train_test_split(
data.drop("survived", axis=1),
data['survived'],
test_size=0.3,
random_state=0,
)

c = pd.crosstab(y_train, X_train['sex'])
chi2_contingency(c)
c

chi_ls = []
for column in X_train.columns:
        c = pd.crosstab(y_train, X_train[column])
        pvalue =    chi2_contingency(c)[1]
        chi_ls.append(pvalue)
selected = pd.Series(chi_ls,
index=X_train.columns).sort_values(ascending=True)[0:4].index
print(selected)
```

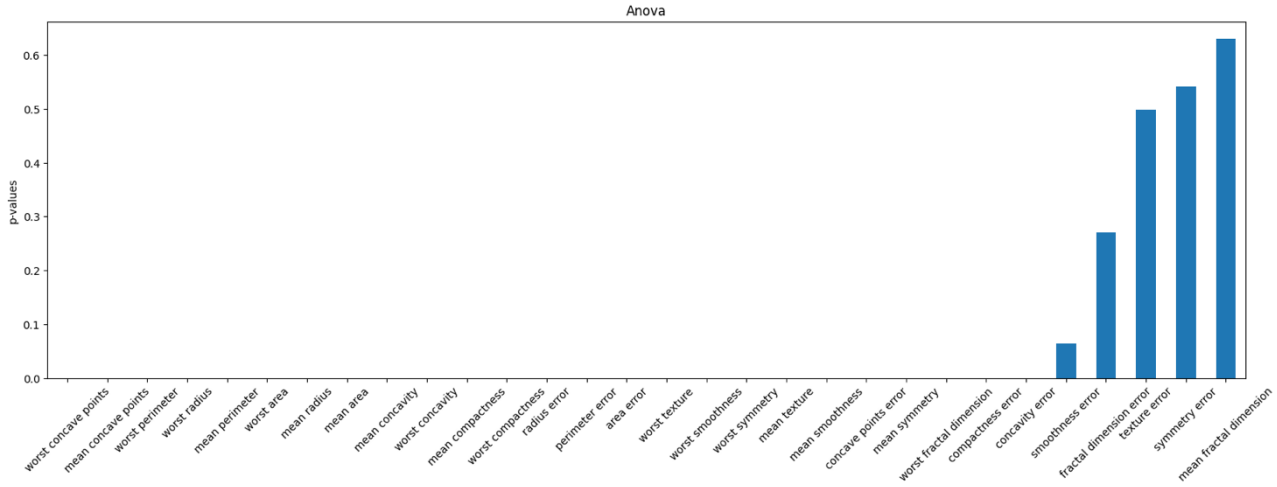


2. ANOVA

- ANOVA atau Analysis of Variance digunakan sebagai teknik seleksi fitur ketika fitur bersifat kontinu dan target bersifat kategorik.

- Dalam one-way ANOVA, langkah pertama adalah menghitung p-value untuk setiap fitur.
  - Nilai p-value menunjukkan kemungkinan bahwa rata-rata suatu fitur sama di berbagai kelas target.
  - Semakin kecil p-value, semakin besar perbedaan distribusi antar kelas, sehingga fitur tersebut dianggap lebih penting terhadap output.

- Fitur kemudian diberi peringkat berdasarkan p-value dan dipilih fitur dengan peringkat tertinggi.




```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import (
    f_classif,
    SelectFpr,
    SelectKBest,
)
from sklearn.model_selection import train_test_split

breast_cancer = load_breast_cancer()
X = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)
y = breast_cancer.target
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

univariate = f_classif(X_train, y_train)
print(univariate)

univariate = pd.Series(univariate[1])
univariate.index = X_train.columns
univariate.sort_values(ascending=True).plot.bar(figsize=(20, 6), rot=45)
plt.ylabel("p-values")
plt.title("Anova")
```

```
sel = SelectFpr(f_classif, alpha=0.05).fit(X_train, y_train)
X_train.columns[sel.get_support()]

X_train_t = sel.transform(X_train)
X_test_t = sel.transform(X_test)
X_train_t = pd.DataFrame(X_train_t, columns=sel.get_feature_names_out())
X_test_t = pd.DataFrame(X_test_t, columns=sel.get_feature_names_out())

sel = SelectKBest(f_classif, k=10).fit(X_train, y_train)
X_train.columns[sel.get_support()]
```

3. Korelasi

- Uji korelasi banyak digunakan dalam data science, terutama pada model regresi linear, untuk mengukur hubungan antara dua peubah.

- Dalam seleksi fitur, metode ini digunakan ketika fitur dan target sama-sama bersifat kontinu (numerik).

- Korelasi membantu memilih sebagian fitur terbaik serta mengidentifikasi dan menghapus fitur yang redundan.

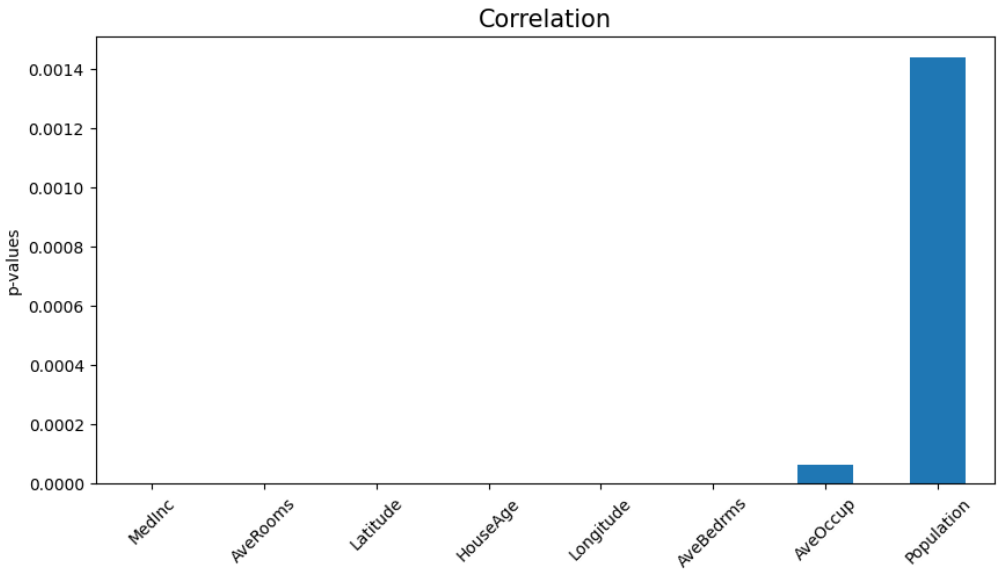
- Salah satu ukuran yang umum digunakan adalah koefisien korelasi Pearson (R), yang bernilai antara -1 hingga 1 dan menunjukkan arah serta kekuatan hubungan antar peubah. Fitur kemudian diberi peringkat berdasarkan nilai absolut koefisien korelasi, dan fitur dengan peringkat tertinggi dipilih.
  - Atau dapat dilakukan uji korelasi antar fitur dengan target, yang kemudian dipilih fitur-fitur yang memiliki p-value kecil

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.feature_selection import (
    f_regression,
    SelectPercentile,
)
from sklearn.model_selection import train_test_split

X, y = fetch_california_housing(return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
random_state=0)

univariate = f_regression(X_train, y_train)
print(univariate)

plt.rc("axes", titlesize=15) #fontsize of the title
univariate = pd.Series(univariate[1])
univariate.index = X_train.columns
univariate.sort_values(ascending=True).plot.bar(figsize=(10, 5),
rot=45)
plt.ylabel("p-values")
plt.title("Correlation")
plt.show()
```




```
sel = SelectPercentile(f_regression,
percentile=30).fit(X_train, y_train)
sel.get_feature_names_out()
```

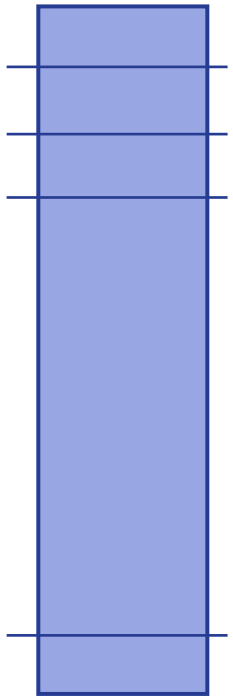
Filtering selection variables in text analytics

- Seleksi fitur dengan metode filter dalam analisis teks merupakan langkah prapemrosesan yang penting untuk mengurangi data teks berdimensi tinggi (misalnya ribuan kata unik atau N-gram) menjadi himpunan bagian yang lebih kecil dan lebih informatif.

- Metode filter ini menilai relevansi fitur berdasarkan sifat statistik intrinsiknya tanpa menggunakan model pembelajaran mesin tertentu. Hal ini membuatnya lebih efisien secara komputasi, terutama untuk analisis teks dalam skala besar.

Ide dasar

Preprocessing text



Evaluasi,
Lalu lakukan
seleksi

Token 1	Token 2					Token p

Generate target
(misal sentiment)

Generate matriks per token

Metode

- Chi-suared test → Metode ini mengevaluasi hubungan antara istilah tertentu (fitur) dan kategori, serta menentukan apakah kemunculan suatu kata bersifat independen terhadap kelasnya atau tidak.

- ANOVA (Variance threshold) → metode yang menghapus fitur dengan varians sangat rendah (misalnya kata yang muncul dengan frekuensi hampir sama di semua dokumen), karena fitur tersebut memiliki daya pembeda yang rendah.

- Document Frequency (DF) → metode sederhana dan efektif yang menghapus kata dengan frekuensi rendah (kata yang jarang muncul) atau sangat tinggi (kemungkinan merupakan stop word), dengan asumsi bahwa kata yang jarang muncul cenderung kurang informatif.

- Term Frequency (TF) / TF-IDF → metode yang mengidentifikasi kata berdasarkan seberapa sering kata tersebut muncul dalam suatu dokumen dibandingkan dengan seluruh korpus, sehingga lebih memprioritaskan kata-kata yang bersifat diskriminatif.

Ilustrasi

Filtering dengan Document Frequency (DF)

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    'pengiriman cepat dan layanan pelanggan sangat baik',
    'layanan pelanggan buruk dan respon sangat lambat',
    'barang bagus tapi pengiriman sangat lambat',
    'sangat puas dengan pengiriman dan layanan'
]

vectorizer = CountVectorizer(min_df=2, max_df=0.75)
X = vectorizer.fit_transform(corpus)

print("Fitur setelah DF Filtering:",
vectorizer.get_feature_names_out())
```

min_df=2 (kata harus muncul di minimal 2 dokumen)

max_df=0.75 (kata yang muncul di >75% dokumen akan dibuang, misal: 'sangat')

https://colab.research.google.com/drive/1VDVKup7DPwzpP7IMEXLhoyQq7-W7mFfm?usp=sharing

Filtering dengan TF-IDF

TF-IDF memberikan bobot tinggi pada kata yang sering muncul di satu dokumen tetapi jarang di dokumen lain, membantu menyaring kata kunci yang paling relevan secara otomatis.

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# Inisialisasi TfidfVectorizer
tfidf_vec = TfidfVectorizer()
tfidf_matrix = tfidf_vec.fit_transform(corpus)

# Konversi ke DataFrame agar mudah dibaca
df_tfidf = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=tfidf_vec.get_feature_names_out()
)

print("\nSkor TF-IDF (Semakin tinggi skor, semakin penting kata tersebut):")
print(df_tfidf.head())
```

- Perbedaan Utama:
  - DF Filtering bersifat "hard" (menghapus fitur sepenuhnya dari dataset). Biasanya digunakan di awal untuk mengurangi beban komputasi.
  - TF-IDF bersifat "soft" (memberikan pembobotan). Kata-kata yang tidak informatif akan memiliki skor mendekati nol, sehingga pengaruhnya kecil saat masuk ke model Machine Learning.

Lalu, lakukanlah metode filtering untuk seleksi fitur pada fitur-fitur data scraping yang sudah dikumpulkan oleh masing-masing kelompok

Terima Kasih

IPB University

Bogor Indonesia

Inspiring Innovation with Integrity

in Agriculture, Ocean and Biosciences for a Sustainable World